面向 Agent 平台的版本一致安全执行

Version-Consistent Safety Enforcement for Agent Platforms

中文研究初稿　2026 年 9 月 23 日

研究状态。本文提出待验证的 Relay 与 Guard 联合执行协议。当前两个产品分别具有若干相关机制，但尚未实现本文定义的联合执行上下文。文中的引理针对抽象状态机；尚未运行模型检查，也未获得生产实现的端到端证明。

本稿适合用于确定研究问题、证明义务与实验设计。正式投稿前须补齐可执行模型、代码符合性证据和可复现实验。

摘要

Agent 平台允许开发者修改定义、针对某一版本运行评估、发布不可变制品，并在异步部署中处理持续到来的请求。保护策略与运行授权又可能独立变化。由此产生一个安全问题：已通过的评估是否仍对应当前发布内容；一个请求是否会混用不同保护版本；授权撤销是否会影响尚未执行的外部操作；流式文本是否会在检查前抵达用户。本文提出一个版本化执行上下文，将 Agent 内容摘要、运行实例和 Guard 保护发布绑定到一次调用，同时在每次外部操作的准入点读取当前授权。我们定义发布证据新鲜性、单次调用的保护版本一致性、外部操作准入授权和流式交付边界四类安全性质，并对全缓冲模式的“检查前不交付”性质给出抽象状态机上的归纳证明。验证计划包括有限状态模型检查、代码轨迹符合性检查、故障注入以及安全性与性能的分离评估。所有保证均以完整拦截和可信执行组件为前提；本文不声称模型输出在语义上必然正确，也不声称检测器没有漏报。

关键词：Agent 平台；形式化方法；版本一致性；运行时准入；策略发布；流式安全。

1 引言

Agent 平台存在多个相互独立的时间轴。开发者可能在一次成功评估后再次编辑 Agent；控制面可能在旧请求仍在执行时发布新策略；管理员可能在工具调用进行中撤销授权；模型可能在检测器获得完整上下文前连续输出文本。若把“已发布”或“已保护”这样的界面状态直接解释成运行时安全结论，就会掩盖这些竞态。

本文研究的问题是：**在什么明确前提下，平台能够保证每次获准执行的外部操作与每段交付给用户的输出，都能对应到确定的 Agent 版本、保护发布和授权决定？** 我们关注平台边界的可验证性，而不尝试证明语言模型的推理过程总是正确。

论文拟形成三项贡献。第一，定义跨评估、发布、实例化、策略激活和请求执行的版本化控制协议。第二，给出可由状态机验证的安全性质，并将“配置版本固定”与“授权实时检查”区分。第三，设计从抽象模型到真实 Relay、Guard 执行轨迹的符合性验证方法。本文是研究初稿；这些贡献尚未全部实现或验证。

2 系统现状与研究边界

Relay 已有将发布测试结果绑定到 Agent 内容摘要的门禁，并区分不可变 Version 与显式创建的 Instance[4]。Guard 已有编译制品签名、Runner 验签及原子激活，并对流式输出定义全缓冲、窗口缓冲和可中断交付模式[5]。这些是**分别存在的局部机制**，不足以推出两个产品组成的平台已具备联合形式化保证。

表1区分当前机制与本文新增的研究对象。仓库事实来自 2026 年 9 月 23 日的本地检出；正式发表时应将源代码定位固定到可公开访问的提交。

2.1 威胁模型与可信前提

攻击者可以控制提示、检索文档、模型输出、工具参数和请求时序，并利用消息延迟、重试、副本更换和并发发布。我们不假设 Agent 会遵循自然语言安全提示。初始模型信任控制面的事务边界、摘要计算、签名验证、执行网关以及阻止旁路访问的隔离机制；假设哈希抗

表 1: 当前机制与待验证的联合约束

方面	当前可据代码或协议检查的机制	研究中新增的约束
发布证据	Relay 检查评估摘要与当前内容摘要是否相同	固定评估器、数据集版本，并追踪至真实运行实例
保护发布	Guard 制品签名，Runner 按发布激活	将具体 Guard 发布与每次 Agent 调用绑定
授权	Relay 有项目范围内的能力及资源关系判断	每次外部操作完整经过准入；定义撤销与在途操作语义
输出交付	Guard 定义流式检查响应与交付字段	证明实际网关只交付获准字节且没有旁路

碰撞、签名和限定范围的凭据不可伪造。集群管理员失陷、宿主机失陷及侧信道不在第一篇论文的保证范围内。

完整拦截是核心前提：所有相关外部效果和所有客户端可见字节必须经过所建模的准入或交付接口。若某个 Supervisor、MCP 工具、A2A 路径或直接网络请求可绕过接口，本文结论不覆盖该路径。隔离是否成立需要部署配置与攻击性测试的独立证据，不能由抽象模型代替。

3 形式化模型

设 P, I, V, R, C 分别为项目、实例、Agent 版本、Guard 保护发布和调用的集合。版本 $v \in V$ 包含不可变内容摘要 $\mathrm{digest}(v)$ 及其评估证据引用。实例 $i \in I$ 属于一个项目并指向一个版本。保护发布 $r \in R$ 标识签名制品集合与路由修订。调用 $c \in C$ 在开始时获得固定上下文

$$\kappa(c) = \big(p,\, i,\, v,\, \mathrm{digest}(v),\, r,\, \rho\big), \tag{1}$$

其中 ρ 是路由修订。式(1)是**拟新增的协议对象**，并非当前 Relay 或 Guard 的现成 API。

授权状态 A_t 随时间变化。保护发布在一次调用内固定，以保证可复现；外部操作则在其准入点依据 A_t 重新判断。若研究目标扩展为“撤销后任何远端效果都不会发生”，远端服务必须再次验证有期限或带授权纪元的能力凭据。单次控制面预检查无法撤回已被外部系统接受的操作。

抽象状态包括草稿、评估记录、已发布版本、实例、签名保护发布、Runner 激活状态、调用、工具准入、输出缓冲和已交付字节。主要状态转换为：

```
Edit, Evaluate, PublishVersion, CreateInstance,
ActivateGuardRelease, StartCall, AdmitTool,
AppendOutput, CheckOutput, Deliver, Revoke.
```

模型还包含故障和重试转换。版本构建与副本激活是不同转换；控制面期望的发布不意味着所有 Runner 已收敛。

3.1 拟验证的安全性质

P1：发布证据新鲜。 每个已发布版本都有同一内容摘要上的通过证据。更强的研究版本要求评估套件与评估器也具有固定版本。当前 Relay 的门禁主要验证内容摘要，不能把下式整体视为已实现。

$$\begin{aligned}\mathrm{Published}(v) \implies \mathrm{Passed}(e_v) \wedge \mathrm{Digest}(e_v) = \mathrm{digest}(v) \\ \wedge\ \mathrm{Suite}(e_v) = \mathrm{RequiredSuite}(v).\end{aligned} \tag{2}$$

P2：单次调用的保护版本一致。 一次调用使用的每个 Guard 决定均来自调用开始时固定的发布。若某 Runner 无法服务该发布，应失败而不能默默改用“最新版本”。

$$\forall d \in \mathrm{Decisions}(c), \quad \mathrm{Release}(d) = \mathrm{GuardRelease}(\kappa(c)). \tag{3}$$

P3：外部操作先授权。 每个获准执行的外部操作在其准入线性化点，具有与项目、实例、操作和资源相匹配的许可。

$$\mathrm{Admit}(c, a, x, t) \implies \mathrm{Permit}(A_t, \kappa(c), a, x). \tag{4}$$

式(4)只讨论准入；若要保证远端效果在撤销后绝不发生，还需目标服务参与围栏验证。

P4：受检字节交付。 全缓冲模式在最终检查通过前不交付任何输出。通过后，新增交付字节只能来自该次检查返回的可交付文本，包括获准的替换文本。

$$\mathrm{mode}(c) = \texttt{full_buffered} \wedge \neg\,\mathrm{FinalPassed}(c) \implies \mathrm{Delivered}(c) = \varepsilon. \tag{5}$$

窗口缓冲与可中断模式只能主张每段已释放文本经过相应阶段的检查，不能主张后续上下文可收回先前字节。Guard 协议也明确了这一边界[5]。

跨项目隔离可作为第五条性质，但应先完成所有数据和工具出口的拦截清单。部署最终收敛属于活性性质，需要另行给出公平性与基础设施可用性假设。

3.2 一个可写入正文的归纳证明

考虑全缓冲输出的抽象状态 (b, q, o)，分别表示缓冲文本、最终检查状态和已交付文本。初态满足 $b = o = \varepsilon$ 且 $q = \mathrm{pending}$。`AppendOutput` 只修改 b；`CheckFinal` 将 q 改为 passed 或 blocked，两种结果均为终态；`Deliver` 仅在 $q = \mathrm{passed}$ 时可执行，且只追加本次检查的 `released_text`；故障与阻断转换不追加 o。

引理 1 (抽象协议的检查前不交付). 该状态机的每个可达状态均满足

$$q \in \{\mathrm{pending}, \mathrm{blocked}\} \implies o = \varepsilon.$$

证明. 对执行轨迹长度归纳。初态显然满足命题。`AppendOutput`、故障及阻断转换均不增加 o；`CheckFinal` 也不修改 o。`Deliver` 在 q 为 pending 或 blocked 时不可执行；通过后的状态不能再转为 pending 或 blocked。因此每个允许的转换保持命题。另可证明：检查通过后，o 始终是该检查返回的可交付文本的前缀；这还要求交付转换串行且不重复。 □

此引理只覆盖抽象网关协议。若真实网关在调用 Guard 前已经转发模型分片，或在超时后发送原始文本，该实现就不满足上述转换关系，即使抽象引理本身成立。

4 验证路线与实现映射

有限状态探索。 使用 TLA+ 描述转换，借助 TLC 在有限的项目、版本、Runner 和调用集合上检查 P1 至 P4[3]。故障调度应覆盖发布延迟、NACK、重试、撤销、副本切换、缺失最终分片与响应重复，并保存反例轨迹。有限状态模型检查证明的是所选界限内的模型性质，不是无限状态下的生产代码定理。

代码轨迹符合性。 为真实执行定义以下事件：

```
VersionPublished, InstanceCreated, CallStarted,
GuardDecision, ToolAdmitted, OutputReleased,
CallTerminated.
```

每条事件关联真实实例 ID、调用 ID、Agent 摘要和 Guard 发布 ID。通过真实边界重放确定性场景，并拒绝无法映射为抽象合法转换的执行轨迹。

实施缺口。 在宣称端到端保证前，需要实现并认证式(1)中的联合上下文，贯通真实实例身份，连接 Relay 的模型或 Agent 输出路径与 Guard 执行点，验证网关按 Guard 响应交付，并排查旁路。当前 Relay 的一个评估记录路径把 `agentId` 写入名为 `instanceId` 的字段；它不能作为真实运行实例归因的证据[4]。Relay 的审批判定原语也不能替代尚不存在的完整审批执行流程。

证明升级。 若有限模型稳定，可选择关键不变量进行无界归纳证明。即使得到抽象协议的机器检查证明，也仍需给出实现到模型的精化映射，或者将可信执行核心缩小到可直接核验的代码范围。本文不以“跑过 TLC”替代这一步。

5 实验设计

表 2: 研究问题与证据形式

问题	实验	主要报告指标
RQ1	与每步解析“最新发布”的基线比较；注入并发发布与副本切换	反例轨迹、混用发布的调用比例
RQ2	将模型导出的故障时序在真实 Controller、Runner、网关和 Agent 路径重放	性质违反次数、最终客户端字节、轨迹覆盖范围
RQ3	同一输入及检测器比较无 Guard、全缓冲、窗口缓冲与可中断模式	首个安全字节延迟、完成延迟、内存、吞吐、调用次数
RQ4	用独立审阅的多语言保留集检查内容与工具动作	检出率、误报率、正常任务完成率及置信区间

用例同时包含成功路径与明确故障：过期评估、跨项目资源替换、权限撤销、后段阻断、敏感串跨分片、文本变换、缺失最终分片、乱序、取消、超时、Redis 租约过期和旧发布

请求跨副本继续执行。正式论文需公开脱敏夹具、脚本、模型检查配置、依赖及镜像版本，并列出未覆盖路径。协议正确性与检测器的语义检出率分别报告，不能由前者推断后者。

6 相关工作与论文定位

Agent-C 研究对 Agent 动作施加形式化时序约束，并在生成时约束工具调用[1]。本文拟研究的区别是跨开发证据、异步发布、真实实例身份、动态授权及最终内容交付的平台边界；这只是研究定位，不能在缺乏系统比较时宣称优先性或新颖性。NVIDIA NeMo Guardrails 已公开说明 `stream_first` 在延迟与检查前交付之间的取舍[2]。因此本文的流式部分应以可复现的版本固定、故障处理和客户端字节证据为贡献，而非声称“首次实现流式 Guardrail”。

7 局限与结论

当前可分别核对的机制包括 Relay 的内容摘要发布门禁与范围化授权，以及 Guard 的签名制品、发布激活和流式交付协议。尚未验证的是：两产品联合上下文、所有外部效果与输出的完整拦截、模型到代码的符合性、TLC 检查结果和无界证明。语言模型输出的事实正确性、检测器无漏报、集群不失陷以及旁路路径的保护均不属于本文已成立的结论。

本文的核心方法是把“安全可控”拆成可执行的系统命题：哪份证据允许发布、哪版策略处理请求、哪个授权允许动作、哪些字节可以交付。下一阶段应先交付一份可运行的形式化模型和一条真实联合执行轨迹，再据实修改摘要与结论。

参考文献

[1] A. Kamath 等，Enforcing Temporal Constraints for LLM Agents，2025。https://arxiv.org/abs/2512.23738。

[2] NVIDIA，Output Rail Streaming。https://docs.nvidia.com/nemo/guardrails/latest/configure-rails/yaml-schema/streaming/output-rail-streaming.html。

[3] TLA+ Foundation，TLA+ Tools and TLC。https://github.com/tlaplus/tlaplus。

[4] TaskLattice Relay，本地仓库快照，`0670ae6`，2026 年 9 月 22 日。相关文件：`apps/control/server/expert-agents/expert-agent-domain.ts`、`apps/control/server/expert-agents/expert-agent-validation-service.ts`、`docs/agent-developer-architecture.md`。https://github.com/tasklattice/tasklattice-relay。

[5] TaskLattice Guard，本地仓库快照，`0539f0f`，2026 年 9 月 22 日。相关文件：`docs/architecture.md`、`docs/gateway-integration.md`、`runner/artifact_store.py`。https://github.com/tasklattice/tasklattice-guard。